

T2: Some more Sorting Algorithms in C

This tutorial introduces four sorting algorithms: Bubble Sort, Insertion Sort, Quick Sort, and Merge Sort. You'll fill in the control structures based on the provided comments to understand how these algorithms work.

- ❖ You can use these [animations](#) as a reference to help complete the algorithms provided below.

1. Bubble Sort

Bubble Sort repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. The process repeats until no more swaps are needed.

```
#include <stdio.h>

void bubbleSort(int arr[], int n) {
    int i, j, temp;

    // Outer loop to iterate through each element
    ____ (i = 0; ____ ) {
        // Inner loop for pairwise comparisons
        ____ (j = 0; ____ ) {
            // Compare the adjacent elements
            if (arr[j] > arr[j + 1]) {
                // Swap if they are in the wrong order
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(arr)/sizeof(arr[0]);
    bubbleSort(arr, n);
    printf("Sorted array: \n");
    for (int i = 0; i < n; i++) {
```

```

        printf("%d ", arr[i]);
    }
    return 0;
}

```

Hints:

- The outer loop should run from the first element to the second last element.
- The inner loop should run from the first element to the $n-i-1$ element.

1.1 Modified Bubble Sort

In the modified Bubble Sort, we introduce a flag to monitor whether any swaps were made during a pass. If no swaps are made, the array is already sorted, and the algorithm can terminate early.

```

#include <stdio.h>
void bubbleSort(int arr[], int n) {
    int i, j, temp;
    int flag;

    // Outer loop to iterate through each element
    ____ (i = 0; ____ ) {
        flag = 0; // Reset swapped flag

        // Inner loop for pairwise comparisons
        ____ (j = 0; ____ ) {
            // Compare the adjacent elements
            if (arr[j] > arr[j + 1]) {
                // Swap if they are in the wrong order
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;

                // Set swapped flag to indicate a swap occurred
                flag = 1;
            }
        }

        // If no two elements were swapped in the inner loop, break
        if (flag == 0) {

```

```

        _____;
    }
}

int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(arr)/sizeof(arr[0]);
    bubbleSort(arr, n);
    printf("Sorted array: \n");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    return 0;
}

```

Hint:

- After each pass, check if **swapped** is **0** (meaning no swaps were made) and break out of the loop if true, as the array is already sorted.

Solution [Modified Bubble Sort]

```

#include <stdio.h>

void bubbleSort(int arr[], int n) {
    int i, j, temp;

    // Outer loop to iterate through each element
    for(i = 0; i < n; i++) {
        int flag = 0;
        // Inner loop for pairwise comparisons
        for(j = 0; j < n-1 -i; j++) {
            // Compare the adjacent elements
            if (arr[j] > arr[j + 1]) {
                // Swap if they are in the wrong order
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                flag = 1;
            }
        }
    }
}

```

```
        if(flag == 0)
        {
            break;
        }
    }
}
```

2. Insertion Sort

Insertion Sort builds the sorted array one item at a time by picking elements from the unsorted part and inserting them into the correct position in the sorted part.

```
#include <stdio.h>

void insertionSort(int arr[], int n) {
    int i, key, j;

    // Loop through each element in the array
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;

        // Move elements greater than key to one position ahead
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}

int main() {
    int arr[] = {12, 11, 13, 5, 6};
    int n = sizeof(arr)/sizeof(arr[0]);
    insertionSort(arr, n);
    printf("Sorted array: \n");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    return 0;
}
```

```
}
```

Hints:

- The outer loop should start from the second element (**i = 1**).
 - The inner loop should shift elements to the right as long as they are greater than the key.
-

3. Quick Sort

Quick Sort is a divide-and-conquer algorithm that picks a pivot element, partitions the array into subarrays, and recursively sorts the subarrays.

```
#include <stdio.h>

void swap(int* a, int* b) {
    int t = *a;
    *a = *b;
    *b = t;
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // Pivot element
    int i = (low - 1);

    // Loop through the array and partition based on pivot
    while (int j = low; j < high) {
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

        // Recursively sort the partitions
    }
}
```

```

        quickSort(arr, ____);
        quickSort(arr, ____);
    }
}

int main() {
    int arr[] = {10, 7, 8, 9, 1, 5};
    int n = sizeof(arr)/sizeof(arr[0]);
    quickSort(arr, 0, n - 1);
    printf("Sorted array: \n");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    return 0;
}

```

Hints:

- The loop should iterate from **low** to **high-1**.
- The recursive calls should sort the subarrays before and after the pivot.

4. Merge Sort

Merge Sort is a divide-and-conquer algorithm that divides the array into halves, recursively sorts each half, and merges the sorted halves.

```

#include <stdio.h>

void merge(int arr[], int l, int m, int r) {
    int n1 = m - l + 1;
    int n2 = r - m;
    int L[n1], R[n2];

    // Copy data to temporary arrays
    ____ (int i = 0; i < n1; ____ )
        L[i] = arr[l + i];
    ____ (int j = 0; j < n2; ____ )
        R[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l;

```

```

// Merge the temporary arrays back into arr[l..r]
_____ (i < n1 && j < n2) {
    if (L[i] <= R[j]) {
        arr[k] = L[i];
        i++;
    } else {
        arr[k] = R[j];
        j++;
    }
    k++;
}

// Copy the remaining elements of L[], if any
_____ (i < n1) {
    arr[k] = L[i];
    i++;
    k++;
}

// Copy the remaining elements of R[], if any
_____ (j < n2) {
    arr[k] = R[j];
    j++;
    k++;
}
}

void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;

        // Recursively sort the first and second halves
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);

        // Merge the sorted halves
        merge(arr, l, m, r);
    }
}

int main() {
    int arr[] = {12, 11, 13, 5, 6, 7};

```

```

int arr_size = sizeof(arr)/sizeof(arr[0]);

printf("Given array is \n");
for (int i = 0; i < arr_size; i++) {
    printf("%d ", arr[i]);
}

mergeSort(arr, 0, arr_size - 1);

printf("\nSorted array is \n");
for (int i = 0; i < arr_size; i++) {
    printf("%d ", arr[i]);
}
return 0;
}

```

Hints:

- Use loops to copy data to temporary arrays `L[ ]` and `R[ ]`.
 - The loop for merging should compare elements of `L[ ]` and `R[ ]` and merge them back into the main array.
 - Ensure that any remaining elements in `L[ ]` or `R[ ]` are also copied back to the main array.
-

Exercise Problems

1. Intersection of Two Sorted Arrays

Write a program to find the intersection of two sorted arrays. The intersection should include all elements that appear in both arrays. For example, given `[1, 2, 2, 3, 4]` and `[2, 2, 3, 5]`, the output should be `[2, 2, 3]`.

Hint: Use two pointers to traverse both arrays and find common elements.

```
#include <stdio.h>
```



```
int main()
{
    int arr1[] = {34, 90, 90, 91, 91, 92, 95};
    int arr2[] = {34, 91, 91, 92, 92, 95};

    int n1 = sizeof(arr1) / sizeof(arr1[0]);
    int n2 = sizeof(arr2) / sizeof(arr2[0]);

    int i = 0, j = 0;

    while (i < n1 && j < n2)
    {
        // Skip duplicates in arr1
        if (i > 0 && arr1[i] == arr1[i - 1])
        {
            i++;
            continue;
        }
        // Skip duplicates in arr2
        if (j > 0 && arr2[j] == arr2[j - 1])
        {
            j++;
            continue;
        }
        // Compare elements in both arrays
        if (arr1[i] < arr2[j])
        {
            i++;
        }
        else if (arr1[i] > arr2[j])
        {
            j++;
        }
        else
        {
            printf("%d ", arr1[i]);
            i++;
            j++;
        }
    }

    return 0;
}
```

2. Reversing an Array

Write a program to reverse the elements of an array. This exercise involves manipulating the positions of elements, similar to what happens during sorting.

Hint: Swap elements starting from the ends of the array moving towards the center.

```
#include <stdio.h>

int main()
{
    int arr[] = {34, 90, 91, 92, 95};
    int n = sizeof(arr) / sizeof(arr[0]);
    int i = 0;

    for (i = 0; i < n/2; i++)
    {
        int temp;
        temp = arr[i];
        arr[i] = arr[n - i - 1];
        arr[n - i - 1] = temp;
    }

    for(i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
    return 0;
}
```

3. Rotating an Array

Write a program to rotate an array by k positions to the right. For example, if the array is `[1, 2, 3, 4, 5]` and $k = 2$, the result should be `[4, 5, 1, 2, 3]`.

Hint: Break the array into two parts and swap them or reverse the parts individually and then the whole array.

4. Sorting an Array of Strings Based on Length

Write a program to sort an array of strings based on their lengths. For example, given `["apple", "banana", "kiwi", "plum"]`, the sorted array should be `["kiwi", "plum", "apple", "banana"]`.

Hint: Implement a custom comparison function that compares the lengths of the strings.

5. Finding the Largest Number Formed by Array Elements

Given an array of non-negative integers, arrange them such that they form the largest possible number. For example, given `[3, 30, 34, 5, 9]`, the largest number formed is `9534330`.
